Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА № 3

«Функциональные возможности языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-31Б _____ (Арифулов Р.Х.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):
- Балльная оценка:
- Оценка:

2026 г.

1 Цель работы

Изучение возможностей функционального программирования в языке Python.

2 Общее описание задания

Задание лабораторных работ № 3–4 состоит из решения нескольких задач. В данной работе решены задачи 1–4, задачи 5–7 выполнены в лабораторной работе № 4.

Файлы, содержащие решения отдельных задач, располагаются в пакете lab_python_fp. Решение каждой задачи располагается в отдельном файле. При запуске каждого файла выдаются тестовые результаты выполнения соответствующего задания.

3 Задача 1 (файл field.py)

Необходимо реализовать генератор field, который последовательно выдает значения ключей словаря. В качестве первого аргумента генератор принимает список словарей, дальше через *args генератор принимает неограниченное количество аргументов.

Если передан один аргумент, генератор последовательно выдает только значения полей, если значение поля равно None, то элемент пропускается. Если передано несколько аргументов, то последовательно выдаются словари, содержащие данные элементы. Если поле равно None, то оно пропускается. Если все поля содержат значения None, то пропускается элемент целиком.

Файл field.py:

```
def field(items, *args):
    assert len(args) > 0
    for item in items:
        if len(args) == 1:
            value = item.get(args[0])
            if value is not None:
                yield value
        else:
            result = {key: item.get(key) for key in args if item.get(key) is
not None}
            if result:
                yield result
```

```
if __name__ == '__main__':
    goods = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'color': 'black'},
        {'title': None, 'price': None, 'color': 'white'},
        {'title': 'Стеллаж', 'price': 7000}
    ]
    print(list(field(goods, 'title')))
    print(list(field(goods, 'title', 'price')))
    print(list(field(goods, 'price', 'color')))
```

Результат выполнения программы приведен на рисунке 1.

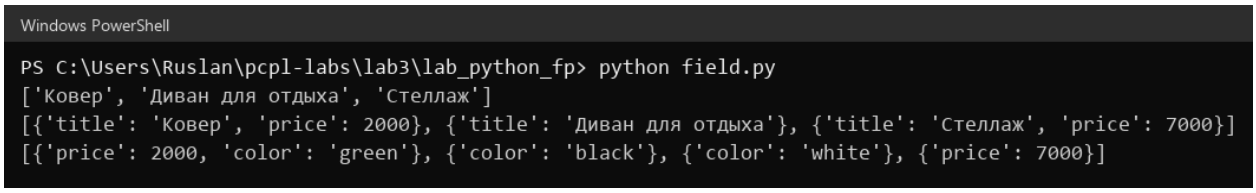


Рисунок 1 — Результат выполнения field.py

4 Задача 2 (файл gen_random.py)

Необходимо реализовать генератор gen_random(количество, минимум, максимум), который последовательно выдает заданное количество случайных чисел в заданном диапазоне от минимума до максимума, включая границы диапазона.

Файл gen_random.py:

```
import random


def gen_random(num_count, begin, end):
    for i in range(num_count):
        yield random.randint(begin, end)


if __name__ == '__main__':
    print(list(gen_random(5, 1, 3)))
    print(list(gen_random(10, 1, 100)))
```

Результат выполнения программы приведен на рисунке 2.

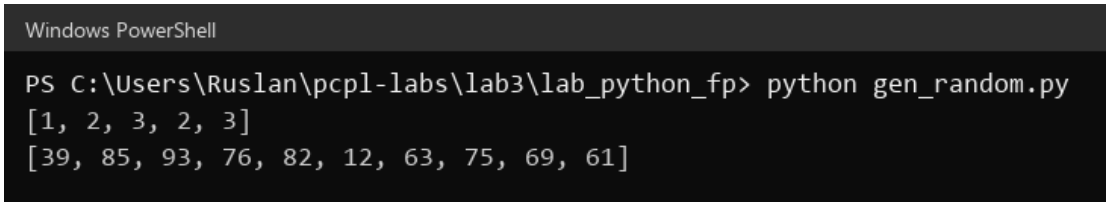


Рисунок 2 — Результат выполнения gen_random.py

5 Задача 3 (файл unique.py)

Необходимо реализовать итератор Unique(данные), который принимает на вход массив или генератор и итерируется по элементам, пропуская дубликаты. Конструктор итератора также принимает на вход именованный bool-параметр ignore_case, в зависимости от значения которого будут считаться одинаковыми строки в разном регистре. По умолчанию этот параметр равен False. При реализации необходимо использовать конструкцию **kwargs. Итератор должен поддерживать работу как со списками, так и с генераторами, и не должен модифицировать возвращаемые значения.

Файл unique.py:

```
from gen_random import gen_random


class Unique(object):
    def __init__(self, items, **kwargs):
        self.items = iter(items)
        self.ignore_case = kwargs.get('ignore_case', False)
        self.seen = set()

    def __next__(self):
        while True:
            item = next(self.items)
            key = item
            if self.ignore_case and isinstance(item, str):
                key = item.lower()
            if key not in self.seen:
                self.seen.add(key)
                return item

    def __iter__(self):
        return self


if __name__ == '__main__':
    data = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
    print(list(Unique(data)))

    data = gen_random(10, 1, 3)
    print(list(Unique(data)))

    data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
    print(list(Unique(data)))
    print(list(Unique(data, ignore_case=True)))
```

Результат выполнения программы приведен на рисунке 3.

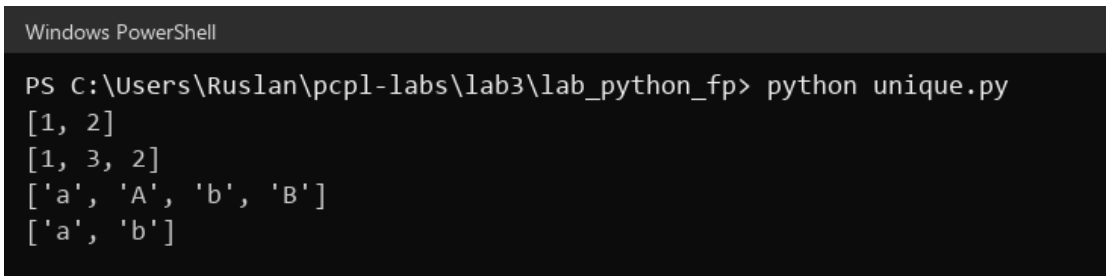


Рисунок 3 — Результат выполнения unique.py

6 Задача 4 (файл sort.py)

Дан массив, содержащий положительные и отрицательные числа. Необходимо одной строкой кода вывести на экран массив, который содержит значения исходного массива, отсортированные по модулю в порядке убывания. Сортировку необходимо осуществлять с помощью функции sorted. Задачу необходимо решить двумя способами: с использованием lambda-функции и без использования lambda-функции.

Файл sort.py:

```
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]

if __name__ == '__main__':
    result = sorted(data, key=abs, reverse=True)
    print(result)

    result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True)
    print(result_with_lambda)
```

Результат выполнения программы приведен на рисунке 4.

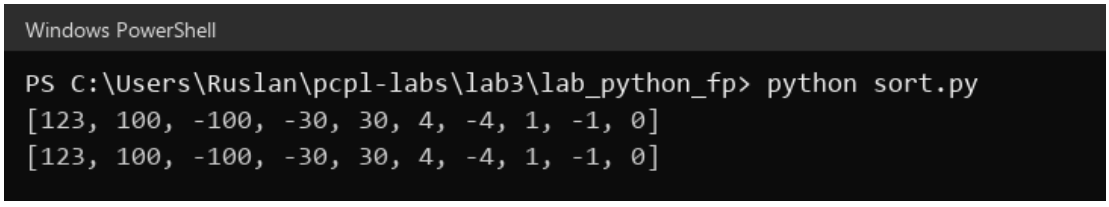


Рисунок 4 — Результат выполнения sort.py